3522 Bulut Bilişim

Akıllı Şehir Yönetim Platformu

Video Akışı ve Akıllı Şehir Kamera Analizi Mikroservisi

Kamera akışı, canlı şehir sahnesi, analiz eventleri, Rekognition stored video hattı ve DynamoDB event store kapsamı

Alan	Değer
Öğrenci	Ahmet Emre Özcan
Öğrenci No	21290560
Ders	3522 Bulut Bilişim
Öğretim Elemanı	Enver Bağcı
Frontend Repo Linki	https://github.com/eemrezcan/bulutproje-final-frontend
Video Analysis Service Repo Linki	https://github.com/eemrezcan/bulutproje4-video-analysis-service
Demo Video Linki	https://youtu.be/5lllkAunq5A

Kapak proje özeti

Alan	Özet
Kapsam	Proje 6 - Video Akışı ve İşleme Uygulaması; Kamera akışı, canlı şehir sahnesi, analiz eventleri, Rekognition stored video hattı ve DynamoDB event store kapsamı
AWS odağı	ECS Fargate service, ECR repository, S3 video bucket, Rekognition Video
Mikroservis sınırı	Ayrı API, ayrı veri modeli, ayrı deployment ve dar IAM yetki sınırı

Özet

Bu rapor, Akıllı Şehir Yönetim Platformu içindeki Video Analysis Service mikroservisini ele alır. Servis şehir kameralarını listeler, canlı kamera hissi veren MJPEG akışları sunar ve kişi sayısı, araç sayısı, hareket seviyesi, kalabalık seviyesi ve recognition label gibi analiz eventlerini üretir. AWS tarafında video dosyası S3'te tutulmuş, Rekognition Video stored analysis SNS ve Lambda üzerinden DynamoDB'ye bağlanmıştır. Video dosyaları veritabanında değil object storage üzerinde tutulur.

Anahtar Kelimeler: Video Akışı, Rekognition Video, S3, SNS, Lambda, DynamoDB, ECS Fargate

İçindekiler

1 Giriş

2 Kapsam

3 Mikroservis sınırı ve sistem mimarisi

4 Teknoloji kararları

5 Veri modeli ve API tasarımı

6 Local çalışma ve demo akışı

7 AWS deployment mimarisi

8 Güvenlik, IAM ve veri sahipliği

9 Test, smoke doğrulama ve kanıtlar

10 Ortak platform entegrasyonu

11 Karşılaşılan sorunlar ve çözüm kararları

12 Sonuç

13 Kaynaklar

1. Giriş

Bu rapor, Akıllı Şehir Yönetim Platformu kapsamında geliştirilen Video Analysis Service bileşenini ayrı bir mikroservis olarak ele alır. Platform tek bir yönetim paneli gibi kullanılsa da her mikroservis kendi veri modeline, kendi API yüzeyine, kendi container image'ına ve kendi AWS yetki sınırına sahiptir.

Temel tasarım hedefi, ders kapsamındaki bulut servislerini yalnızca isim olarak kullanmak değil, doğru problemi doğru AWS servisi ile eşleştirmektir. Bu nedenle raporda local demo akışı ile AWS deployment karşılığı birlikte anlatılmıştır.

2. Kapsam

Bu bölüm, Video Analysis Service mikroservisinin proje içindeki sınırını ve 3522 Bulut Bilişim final yönergesindeki Proje 6 - Video Akışı ve İşleme Uygulaması beklentilerini nasıl karşıladığını özetler. Kapsam, yalnızca uygulama ekranını değil; veri giriş kanalını, servis sorumluluğunu, AWS karşılığını ve raporda savunulacak deployment kararlarını birlikte ele alır.

Kapsam maddesi	Projede karşılığı
Akıllı şehir kamera noktaları	Beş şehir bölgesi için ayrı kamera metadata'sı ve canlı görüntü alanı tanımlandı.
Video akışı	ECS üzerindeki MJPEG endpoint ile canlı kamera hissi veren şehir sahnesi yayınlandı.
Video analizi	Kişi sayısı, araç sayısı, hareket seviyesi, kalabalık seviyesi ve label eventleri üretildi.
Bulut karşılığı	S3 üzerindeki video dosyası Rekognition Video stored analysis ile işlendi; SNS + Lambda sonucu DynamoDB'ye yazdı.
Veri saklama	Video dosyası DynamoDB'de değil S3'te; analiz eventleri DynamoDB'de tutuldu.

Kapsam dışında bırakılan noktalar bilinçli sınırlardır. Amaç üretim ortamı ölçeğinde gerçek şehir altyapısı kurmak değil, ilgili bulut servislerini doğru mimari noktaya yerleştirerek çalışan ve anlatılabilir bir mikroservis sistemi sunmaktır.

3. Mikroservis sınırı ve sistem mimarisi

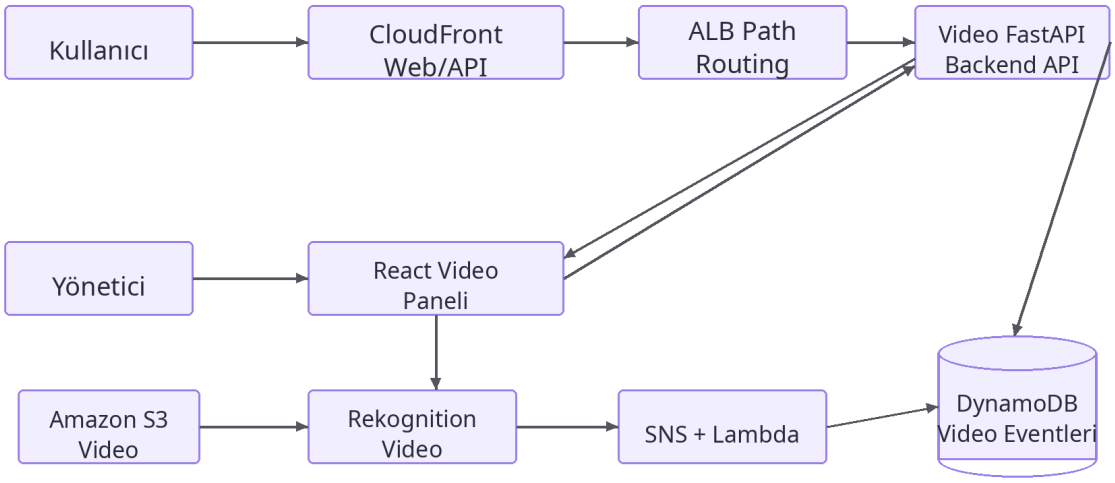
Video Analysis Service aşağıdaki sorumlulukları üstlenir:

- Kamera metadata'sını ve şehir bölgelerini yönetir.
- Canlı kamera hissi veren MJPEG akış endpointleri sunar.
- Kişi sayısı, araç sayısı, hareket ve kalabalık seviyesi eventleri üretir.
- Video analiz sonuçlarını kendi DynamoDB tablosunda saklar.

Bu servis özellikle şu görevleri üstlenmez:

- IoT sensör verisi toplamaz.
- Sıcaklık veya hava kalitesi üretmez.
- ML tahmini yapmaz ve ML modelini eğitmez.
- Video dosyasını DynamoDB'de saklamaz.

Ortak platformda frontend yalnızca REST API çağrıları yapar. Servisler birbirinin DynamoDB tablolarına doğrudan bağlanmaz. Bu yaklaşım mikroservis veri sahipliği ilkesini korur ve deployment ortamında IAM yetkilerinin dar tutulmasını sağlar.



Şekil 1. Video Analysis Service mikroservis mimari diyagramı.

4. Teknoloji kararları

Bileşen	Karar ve gerekçe
Python + FastAPI	Kamera, stream ve analiz endpointleri için kullanıldı.
OpenCV / sentetik sahne	Local ve ECS demosunda stabil canlı şehir görüntüsü üretmek için kullanıldı.
MJPEG live endpoint	Browser tarafında canlı akış hissi için live.mjpg görsel akış endpoint'i uygulandı.
Amazon S3	Video kaynak ve Rekognition input dosyalarını saklar.
Rekognition Video + SNS + Lambda	Stored video analysis sonucunu asenkron işleyip DynamoDB'ye yazar.
DynamoDB	Kamera metadata ve zaman damgalı analiz eventleri için kullanılır.

Backend tarafında FastAPI seçimi, hızlı REST API geliştirme, otomatik Swagger dokümantasyonu ve Python ekosistemiyle uyum nedeniyle yapılmıştır. Frontend tarafında React + Vite + TypeScript kullanılmış; canlı görünüm için TanStack Query polling tercih edilmiştir.

5. Veri modeli ve API tasarımı

video_cameras

Alan	Açıklama
camera_id	Partition key; kamera kimliği.
zone	Kameranın bulunduğu şehir bölgesi.
name	Frontend kamera kartında görünen ad.
stream_url	Canlı görüntü endpoint veya stream yolu.
status	active, passive veya maintenance.
latitude / longitude	Harita entegrasyonu için konum.

video_analysis_results

Alan	Açıklama
camera_id	Partition key; analiz eventinin ait olduğu kamera.
timestamp	Sort key; zaman damgalı analiz geçmişi.
zone	Bölge bazlı sorgular için alan.
people_count / vehicle_count	Kişi ve araç sayısı.
motion_level / crowd_level	low, medium, high veya critical sınıflandırması.
recognition_labels	crowd, traffic, vehicle, emergency gibi etiketler.

API yüzeyi

- GET /health
- GET /cameras
- GET /cameras/{camera_id}/stream
- GET /cameras/{camera_id}/live.mjpg
- GET /analysis/latest
- GET /analysis?camera_id={camera_id}&limit=50
- GET /zones/{zone}/analysis?limit=50
- POST /analysis/generate
- POST /analyzer/start
- POST /analyzer/stop

Endpoint listesi frontend'in ihtiyaç duyduğu okuma, durum ve demo kontrol akışlarını karşılar. Veri üretimi, her modülün kendi doğasına uygun giriş kanalından yapılır.

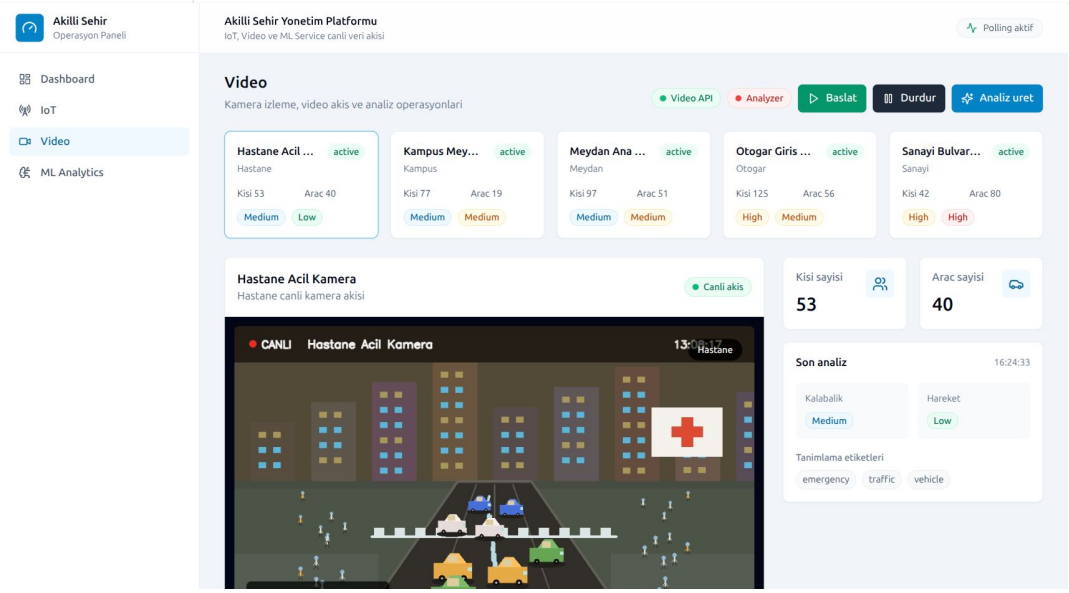
Local-AWS karşılıkları

Local bileşen	AWS karşılığı
Local sentetik video sahnesi	S3 video object ve ECS MJPEG canlı demo endpointi
OpenCV/sentetik analiz	Rekognition Video stored analysis + Lambda result processing
DynamoDB Local	Amazon DynamoDB: video_cameras, video_analysis_results
FastAPI Video container	ECS Fargate Video service
Local stream endpoint	Kinesis Video Streams seçeneği veya CloudFront üzerinden canlı demo yolu

6. Local çalışma ve demo akışı

Local demo Docker Compose ile çalışacak şekilde tasarlanmıştır. Frontend tek paneldir; IoT, Video ve ML servisleri ayrı container süreçleri olarak ayağa kalkar. DynamoDB Local local veritabanı karşılığıdır.

Akış adımı	Açıklama
Başlangıç	Synthetic city scene / video loop
Adım 1	Video Service live MJPEG endpoint
Adım 2	Frontend camera panel
Adım 3	Video analyzer
Adım 4	DynamoDB Local
Adım 5	Video Service REST API
Adım 6	ML ve Dashboard ekranları



Şekil 2. Video Analysis Service canlı deployment ekran görüntüsü.

7. AWS deployment mimarisi

AWS ortamında servis, ayrı ECR repository'si, ayrı ECS Fargate service'i, ayrı task definition ve ayrı IAM task role ile konumlandırılmıştır. Frontend S3 + CloudFront üzerinden yayınlanır; `/api/*` path davranışı ALB'ye yönlendirilir.

AWS akış adımı	Açıklama
Başlangıç	S3 video object
Adım 1	Rekognition Video stored analysis
Adım 2	SNS notification
Adım 3	Lambda: smart-city-final-video-result
Adım 4	DynamoDB: video_analysis_results
Adım 5	ECS Video API
Adım 6	CloudFront frontend
Adım 7	Canlı demo görüntüsü:

AWS akış adımı	Açıklama
Adım 8	ECS Video API -> /api/video/cameras/{id}/live.mjpg -> Frontend

AWS kaynağı	Bu rapordaki görevi
ECS Fargate service	smart-city-final-video-service API ve canlı MJPEG endpointlerini çalıştırır.
ECR repository	smart-city-final/video-service image versiyonlarını saklar.
S3 video bucket	Rekognition input ve video kaynak dosyalarını barındırır.
Rekognition Video	S3 üzerindeki H.264 MP4 için label detection çalıştırır.
SNS + Lambda	Rekognition job tamamlanınca sonucu DynamoDB eventine dönüştürür.
DynamoDB	video_cameras ve video_analysis_results tablolarını barındırır.
CloudFront + ALB	/api/video path'ini Video target group'una yönlendirir.

Bu bölümdeki kaynaklar, local Docker Compose mimarisinin buluttaki karşılığıdır. Amaç backendleri yalnızca bir sunucuya koymak değil; veri girişini, analiz işini, saklama katmanını ve API yüzeyini doğru managed AWS servisleriyle ayırmaktır.

Operasyon ve maliyet notları

Konu	Not
Geçici kapatma	ECS service desired-count=0 yapılarak Fargate task maliyeti durdurulabilir.
Veri koruma	DynamoDB, S3, ECR ve IAM kaynakları silinmeden demo verileri korunur.
ALB notu	ALB'nin stop modu yoktur; tamamen maliyeti kesmek için silmek gerekir, demo öncesi önerilmez.
Yaklaşık maliyet	3 Fargate task + ALB + public IPv4 düşük trafikte yaklaşık 0.13 USD/saat düzeyindedir.

8. Güvenlik, IAM ve veri sahipliği

Her mikroservise yalnızca kendi verisine erişim yetkisi verilmiştir. ECS task execution role sadece ECR image çekme ve CloudWatch Logs yazma gibi altyapı görevleri için; uygulama task role ise servis özelindeki DynamoDB/S3/Lambda erişimleri için kullanılır.

Bu ayrım, mikroservis sınırını hem kod hem deployment seviyesinde görünür hale getirir. Bir servisin diğer modülün tablosuna doğrudan erişmemesi, hata izolasyonu ve ilerde ölçekleme açısından daha sağlıklı bir tasarımdır.

9. Test, smoke doğrulama ve kanıtlar

Kontrol	Sonuç
CloudFront Video API health	/api/video/health status=ok döndürdü.
Canlı stream metadata	stream endpoint mode=live_synthetic_*_mjpeg ve media_type=multipart/x-mixed-replace döndürdü.
Frontend smoke	Video ekranında beş kamera listelendi ve seçili kamera görseli 960x540 yüklendi.

Kontrol	Sonuç
Rekognition job	Stored video analysis SUCCEEDED oldu ve Lambda sonucu DynamoDB'ye yazdı.
ECS service	desired=1, running=1, target health=healthy.

Canlı frontend URL'si https://d1k4ey8nhx4dtb.cloudfront.net adresidir.

Bu doğrulamalar teslim videosunda anlatılacak akış için de kanıt niteliğindedir: önce canlı frontend açılır, ardından modül ekranı gösterilir, en son API ve AWS kaynak eşleşmesi açıklanır.

10. Ortak platform entegrasyonu

Bu mikroservis tek başına anlamlı bir modüldür; ancak platform değeri frontend'de diğer servislerle birlikte görünür. IoT verileri şehir durumunu, video analizleri kamera yoğunluğunu, ML servisi ise risk ve öneri katmanını üretir.

Ortak frontend'in veritabanı yoktur. API Gateway kullanılmamış, bunun yerine CloudFront + ALB path routing ile sade ve gösterilebilir bir deployment kurgusu kurulmuştur.

11. Karşılaşılan sorunlar ve çözüm kararları

Sorun / karar alanı	Çözüm
Demo görüntüsü	Test pattern kaldırıldı; yol, araç, yaya ve CANLI overlay içeren sentetik şehir sahnesi üretildi.
Codec uyumu	Browser uyumu için WebM fallback denendi; final canlı görünümde MJPEG endpoint tercih edildi.
Kinesis kararı	Kinesis doğru AWS karşılığı olarak planlandı; bu deployda canlı demo ECS MJPEG, analiz S3 + Rekognition ile tamamlandı.
DynamoDB seçimi	Video dosyası DB'de tutulmadı; DynamoDB yalnızca zaman damgalı analiz eventleri için kullanıldı.

12. Sonuç

Video Analysis Service, video akışı ve bulut analizini akıllı şehir kamera senaryosunda gösterir. Canlı görüntü ECS MJPEG endpointinden; bulut analizi S3 + Rekognition + SNS + Lambda + DynamoDB hattından sağlanmıştır.

Kaynaklar

Kaynaklar: 3522 Bulut Bilişim final proje yönergesi; AWS resmi dokümantasyonu; FastAPI, React/Vite/TanStack Query/Recharts ve Rekognition Video, SNS, Kinesis Video Streams ve OpenCV dokümantasyonları; proje repository dokümanları.